UNICESUMAR – UNIVERSIDADE CESUMAR

Campus Maringá – Curso de Engenharia de Software

ATIVIDADE DE ESTUDO PROGRAMADA – AEP

2º Bimestre – 2º Semestre de 2026

2ª Entrega – Implementação e Apresentação

PSE EM AÇÃO

IntegraPSE - Sistema de Planejamento e Acompanhamento de Ações de Saúde na Escola
Programa Saúde na Escola (PSE)

Felipe Zotareli Froes

Julia Petrafeza de Oliveira

Vitor Hugo Fernandes Lissoni

Maringá – PR | 2026

SUMÁRIO

1 INTRODUÇÃO....3
2 LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO — LINGUAGEM C....4
2.1 Ambiente de desenvolvimento....4
2.2 Estrutura de dados....4
2.3 Organização em funções....5
2.4 Menu principal....6
2.5 Implementação das funcionalidades....6
2.5.1 Cadastro de ações....6
2.5.2 Listagem e pesquisa de ações....7
2.5.3 Atualização da situação....8
2.5.4 Resumo geral....8
2.6 Validações e tratamento de erros....9
3 ENGENHARIA DE SOFTWARE....10
3.1 Diagrama de casos de uso....10
3.2 Descrição dos casos de uso....10
3.3 Requisitos atualizados....11
3.4 Testes realizados....12
3.5 Acompanhamento das sprints....13
4 PRODUÇÃO TEXTUAL — METEP....14
4.1 Manual do usuário....14
4.1.1 Objetivo do sistema....14
4.1.2 Como executar o programa....14
4.1.3 Opções do menu....14
4.1.4 Observações....14
4.2 Capturas de tela....15
5 CONSIDERAÇÕES FINAIS....20
REPOSITÓRIO GITHUB....21
REFERÊNCIAS....22
APÊNDICE A – CÓDIGO-FONTE COMPLETO DO INTEGRAPSE....23

1 INTRODUÇÃO

Na primeira entrega desta AEP, apresentamos a contextualização do Programa Saúde na Escola (PSE), instituído pelo Decreto nº 6.286, de 5 de dezembro de 2007 (BRASIL, 2007), e realizamos o planejamento do IntegraPSE, uma aplicação em linguagem C destinada a apoiar o registro e o acompanhamento de ações coletivas do programa. Naquela etapa, foram definidos o problema, os objetivos, o escopo, os requisitos, os fluxogramas, os pseudocódigos e o planejamento das sprints.

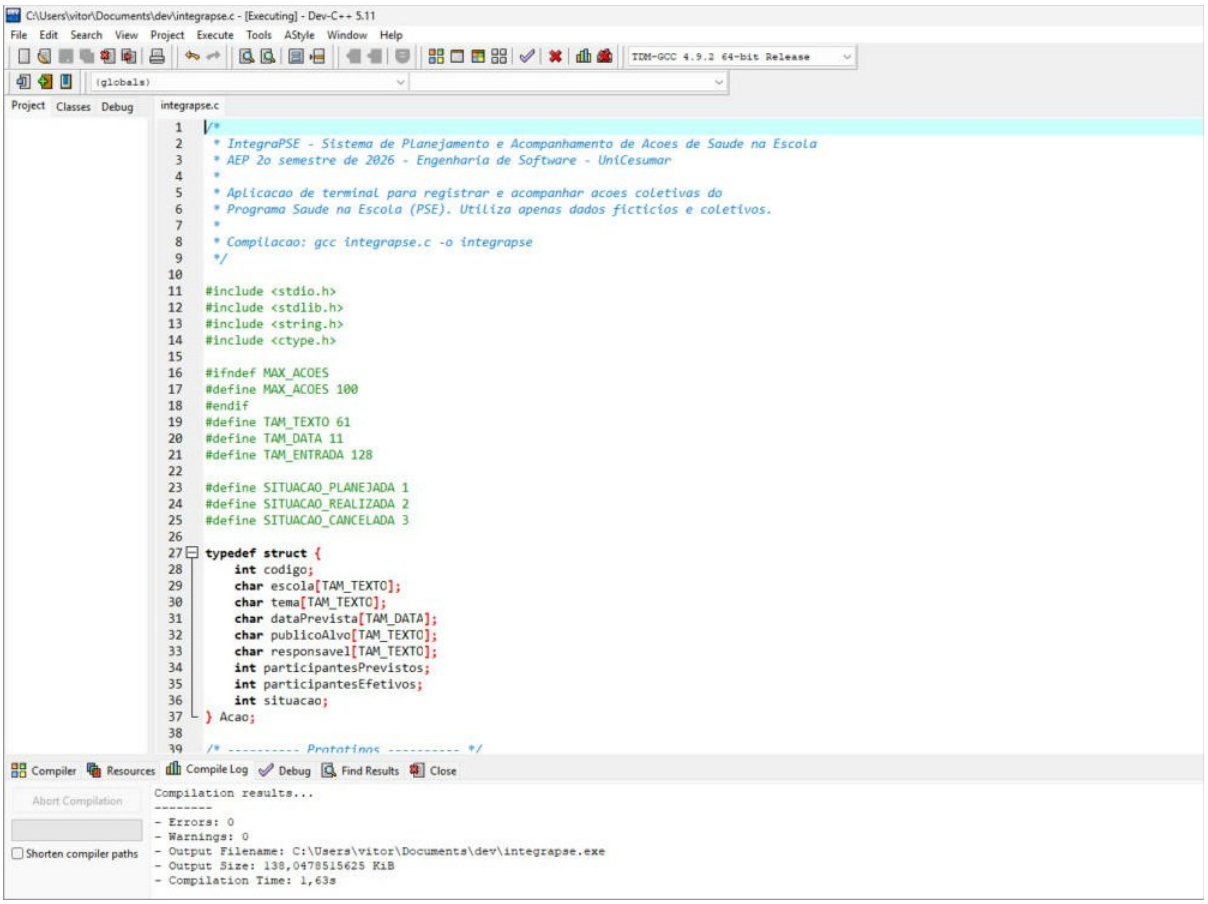
Nesta segunda entrega, apresentamos a implementação da aplicação em linguagem C, o diagrama e a descrição dos casos de uso, os requisitos atualizados, os testes realizados, o manual do usuário, as capturas de tela da execução e as considerações finais do projeto. O código-fonte completo encontra-se no Apêndice A e no repositório do projeto no GitHub.

2 LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO — LINGUAGEM C

2.1 Ambiente de desenvolvimento

O IntegraPSE foi desenvolvido em linguagem C, utilizando o ambiente Dev-C++ 5.11 com o compilador TDM-GCC 4.9.2. Foram utilizadas apenas as bibliotecas padrão stdio.h, stdlib.h, string.h e ctype.h, conforme previsto nos requisitos técnicos da primeira entrega. A Figura 1 apresenta o código-fonte aberto no Dev-C++ e o resultado da compilação, realizada sem erros e sem avisos.

Figura 1 – Código-fonte aberto no Dev-C++ e resultado da compilação



Fonte: Elaborado pelos autores (2026).

Por se tratar de uma aplicação introdutória, todo o código foi mantido em um único arquivo, chamado integrapse.c, armazenado na pasta src do repositório.

2.2 Estrutura de dados

Cada ação do PSE é representada pela estrutura Acao, criada com typedef struct. Os campos da estrutura correspondem às informações definidas na seção 3.3 da primeira entrega. As ações ficam armazenadas em um vetor com capacidade para 100 registros, definida pela constante

MAX_ACOES, e a variável quantidadeAcoes indica quantas posições estão ocupadas. A situação da ação é armazenada como número inteiro: 1 para planejada, 2 para realizada e 3 para cancelada. A Figura 2 apresenta as constantes e a estrutura no Dev-C++.

Figura 2 – Constantes e estrutura Acao no Dev-C++

integrapse.c

```c
#ifndef MAX_ACOES
#define MAX_ACOES 100
#endif
#define TAM_TEXTO 61
#define TAM_DATA 11
#define TAM_ENTRADA 128

#define SITUACAO_PLANEJADA 1
#define SITUACAO_REALIZADA 2
#define SITUACAO_CANCELADA 3

typedef struct {
    int codigo;
    char escola[TAM_TEXTO];
    char tema[TAM_TEXTO];
    char dataPrevista[TAM_DATA];
    char publicoAlvo[TAM_TEXTO];
    char responsavel[TAM_TEXTO];
    int participantesPrevistos;
    int participantesEfetivos;
    int situacao;
} Acao;
```

Fonte: Elaborado pelos autores (2026).

2.3 Organização em funções

O código foi dividido em funções, de modo que cada uma possui uma responsabilidade específica. Cada pseudocódigo apresentado na primeira entrega deu origem a uma função do programa, e algumas funções auxiliares foram criadas para evitar a repetição de código, principalmente na leitura e na validação dos dados. A Tabela 1 apresenta as principais funções.

Tabela 1 – Principais funções do programa

Função	Finalidade
main	Mantém o menu em execução e chama a função correspondente à opção escolhida.
cadastrarAcao	Cadastra uma nova ação após validar os dados informados.
listarAcoes	Exibe todas as ações cadastradas.
pesquisarAcao	Pesquisa ações por código, escola ou tema.
atualizarSituacao	Altera a situação da ação e registra os participantes efetivos.
gerarResumo	Apresenta as quantidades de ações por situação e o percentual de participação.
lerTexto e lerInteiro	Realizam a leitura dos dados digitados e rejeitam entradas inválidas.
dataValida e textoVazio	Verificam a data no formato DD/MM/AAAA e os campos obrigatórios.
buscarPorCodigo	Localiza uma ação pelo código e evita códigos repetidos.

Fonte: Elaborado pelos autores (2026).

2.4 Menu principal

O menu principal foi implementado com uma estrutura do-while e um switch, conforme o pseudocódigo da seção 2.3 da primeira entrega. O menu é exibido repetidamente até que o usuário escolha a opção 0. Caso seja digitada uma opção inexistente ou um texto, o programa apresenta uma mensagem de erro e exibe o menu novamente. A Figura 3 apresenta a função main.

Figura 3 – Função main com o menu principal no Dev-C++

integrapse.c

```c
/* ---------- Funcao principal ---------- */
int main(void) {
    Acao acoes[MAX_ACOES];
    int quantidadeAcoes = 0;
    int opcao;

    do {
        exibirMenu();

        if (!lerInteiro("Escolha uma opcao: ", &opcao)) {
            printf("\n[ERRO] Digite apenas o numero de uma opcao do menu.\n");
            opcao = -1;
            continue;
        }

        switch (opcao) {
            case 1:
                cadastrarAcao(acoes, &quantidadeAcoes);
                break;
            case 2:
                listarAcoes(acoes, quantidadeAcoes);
                break;
            case 3:
                pesquisarAcao(acoes, quantidadeAcoes);
                break;
            case 4:
                atualizarSituacao(acoes, quantidadeAcoes);
                break;
            case 5:
                gerarResumo(acoes, quantidadeAcoes);
                break;
            case 0:
                printf("\nPrograma encerrado. ");
                printf("Os dados desta execucao foram descartados.\n");
                break;
            default:
                printf("\n[ERRO] Opcao invalida. Escolha um numero entre 0 e 5.\n");
        }
    } while (opcao != 0);

    return 0;
}
```

Fonte: Elaborado pelos autores (2026).

2.5 Implementação das funcionalidades

2.5.1 Cadastro de ações

A função cadastrarAcao segue o pseudocódigo CADASTRAR_ACAO. Antes de gravar a ação, o programa verifica se ainda há espaço no vetor, se o código é positivo e não repetido, se os campos obrigatórios foram preenchidos, se a data é válida e se a quantidade de participantes não é negativa. Toda ação cadastrada recebe inicialmente a situação PLANEJADA. A Figura 4 apresenta essa função.

Figura 4 – Função cadastrarAcao no Dev-C++

integrapse.c

```c
/* ---------- Funcionalidades do sistema ---------- */
void cadastrarAcao(Acao acoes[], int *quantidade) {
    Acao nova;
    char data[TAM_ENTRADA];

    printf("\n--- CADASTRAR ACAO ---\n");

    if (*quantidade >= MAX_ACOES) {
        printf("[ERRO] Limite de %d acoes atingido. ", MAX_ACOES);
        printf("Nao e possivel cadastrar.\n");
        return;
    }

    if (!lerInteiro("Codigo da acao (numero positivo): ", &nova.codigo) ||
        nova.codigo <= 0) {
        printf("[ERRO] Codigo invalido. Informe um numero inteiro maior que zero.\n");
        return;
    }
    if (buscarPorCodigo(acoes, *quantidade, nova.codigo) != -1) {
        printf("[ERRO] Ja existe uma acao com o codigo %d.\n", nova.codigo);
        return;
    }

    lerTexto("Escola: ", nova.escola, TAM_TEXTO);
    lerTexto("Tema: ", nova.tema, TAM_TEXTO);
    lerTexto("Data prevista (DD/MM/AAAA): ", data, TAM_ENTRADA);
    lerTexto("Publico-alvo: ", nova.publicoAlvo, TAM_TEXTO);
    lerTexto("Responsavel: ", nova.responsavel, TAM_TEXTO);

    if (textoVazio(nova.escola) || textoVazio(nova.tema) ||
        textoVazio(nova.publicoAlvo) || textoVazio(nova.responsavel)) {
        printf("[ERRO] Existem campos obrigatorios vazios. Cadastro cancelado.\n");
        return;
    }
    if (!dataValida(data)) {
        printf("[ERRO] Data invalida. Use DD/MM/AAAA. Cadastro cancelado.\n");
        return;
    }
    strcpy(nova.dataPrevista, data);
    if (!lerInteiro("Participantes previstos: ", &nova.participantesPrevistos) ||
        nova.participantesPrevistos < 0) {
        printf("[ERRO] Quantidade de participantes invalida. Cadastro cancelado.\n");
        return;
    }

    nova.participantesEfetivos = 0;
    nova.situacao = SITUACAO_PLANEJADA;

    acoes[*quantidade] = nova;
    (*quantidade)++;

    printf("[OK] Acao %d cadastrada com sucesso ", nova.codigo);
    printf("(situacao inicial: PLANEJADA).\n");
}
```

Fonte: Elaborado pelos autores (2026).

2.5.2 Listagem e pesquisa de ações

A função listarAcoes percorre o vetor com uma estrutura for e exibe os dados de todas as ações cadastradas. A função pesquisarAcao permite pesquisar por código, escola ou tema. Na pesquisa por escola ou tema, o usuário pode digitar apenas parte do nome, sem diferenciar letras maiúsculas e minúsculas. Para isso, foi criada a função contemTexto, apresentada na Figura 5.

Figura 5 – Função contemTexto no Dev-C++

integrapse.c

```c
/* Pesquisa por trecho, sem diferenciar letras maiusculas e minusculas. */
int contemTexto(const char *texto, const char *trecho) {
    int i, j;
    int tamTexto = (int)strlen(texto);
    int tamTrecho = (int)strlen(trecho);

    for (i = 0; i <= tamTexto - tamTrecho; i++) {
        for (j = 0; j < tamTrecho; j++) {
            if (tolower((unsigned char)texto[i + j]) !=
                tolower((unsigned char)trecho[j])) {
                break;
            }
        }
        if (j == tamTrecho) {
            return 1;
        }
    }
    return 0;
}
```

Fonte: Elaborado pelos autores (2026).

2.5.3 Atualização da situação

A função atualizarSituacao localiza a ação pelo código, mostra a situação atual e solicita a nova situação. Quando a ação é marcada como realizada, o programa solicita a quantidade de participantes efetivos. Quando a ação é cancelada ou volta a ser planejada, os participantes efetivos são zerados.

2.5.4 Resumo geral

A função gerarResumo segue o fluxograma detalhado da Figura 2 da primeira entrega. O vetor é percorrido uma vez, contando as ações planejadas, realizadas e canceladas e somando os participantes previstos e efetivos das ações realizadas. O percentual de participação só é calculado quando o total previsto é maior que zero, evitando divisão por zero. A Figura 6 apresenta essa função.

Figura 6 – Função gerarResumo no Dev-C++

integrapse.c

```c
void gerarResumo(const Acao acoes[], int quantidade) {
    int i;
    int planejadas = 0, realizadas = 0, canceladas = 0;
    int previstosRealizadas = 0, totalEfetivos = 0;
    float percentual = 0.0f;

    printf("\n--- RESUMO GERAL ---\n");

    if (quantidade == 0) {
        printf("Nenhuma acao cadastrada.\n");
        return;
    }

    for (i = 0; i < quantidade; i++) {
        if (acoes[i].situacao == SITUACAO_PLANEJADA) {
            planejadas++;
        } else if (acoes[i].situacao == SITUACAO_REALIZADA) {
            realizadas++;
            previstosRealizadas += acoes[i].participantesPrevistos;
            totalEfetivos += acoes[i].participantesEfetivos;
        } else if (acoes[i].situacao == SITUACAO_CANCELADA) {
            canceladas++;
        }
    }

    if (previstosRealizadas > 0) {
        percentual = (float)totalEfetivos / previstosRealizadas * 100.0f;
    }

    printf("Total de acoes cadastradas ..............: %d\n", quantidade);
    printf("Acoes planejadas ........................: %d\n", planejadas);
    printf("Acoes realizadas ........................: %d\n", realizadas);
    printf("Acoes canceladas ........................: %d\n", canceladas);
    printf("Participantes previstos (acoes realizadas): %d\n", previstosRealizadas);
    printf("Participantes efetivos ..................: %d\n", totalEfetivos);
    printf("Percentual de participacao ..............: %.1f%%\n", percentual);
}
```

Fonte: Elaborado pelos autores (2026).

2.6 Validações e tratamento de erros

As validações foram concentradas nas funções auxiliares de leitura. As mensagens de erro começam com [ERRO] e as confirmações com [OK], facilitando a compreensão do usuário. O programa trata as seguintes situações:

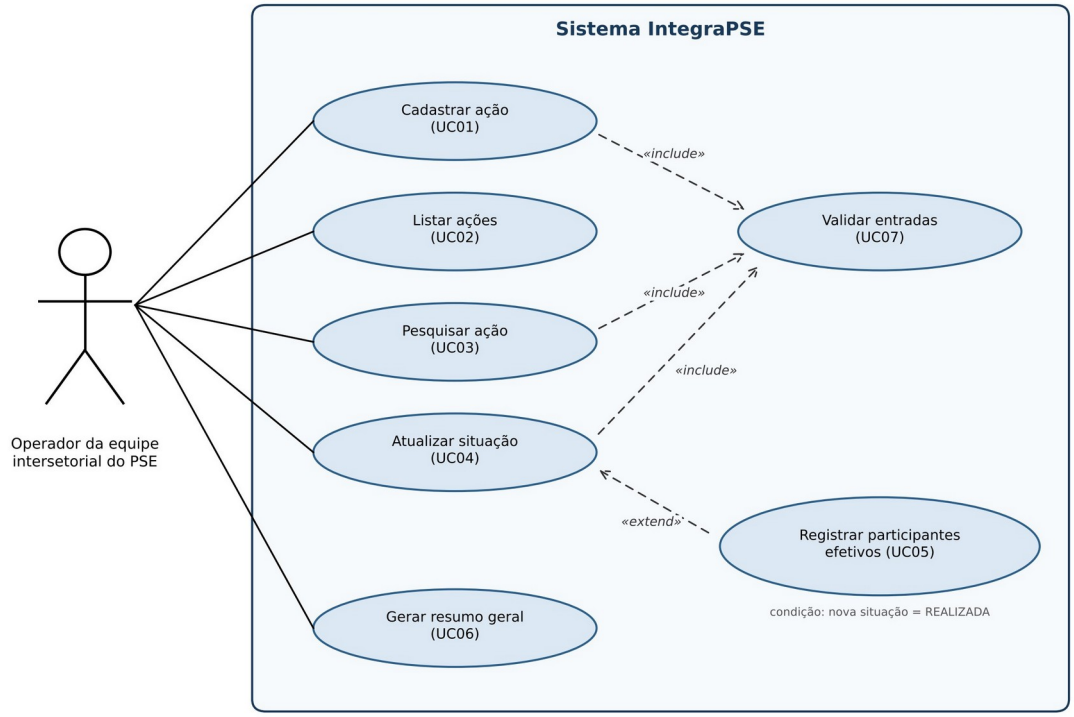
- opção inexistente ou texto digitado no menu;
- código não numérico, menor ou igual a zero ou já cadastrado;
- campos obrigatórios vazios;
- data inexistente ou fora do formato DD/MM/AAAA;
- quantidade de participantes negativa;
- limite de armazenamento atingido;
- pesquisa ou atualização de ação inexistente.

3 ENGENHARIA DE SOFTWARE

3.1 Diagrama de casos de uso

O diagrama de casos de uso representa as funcionalidades do sistema e a interação do usuário com elas (SOMMERVILLE, 2011). No IntegraPSE, há um único ator, o operador da equipe intersetorial, definido na seção 3.1 da primeira entrega. Cada caso de uso foi identificado pela sigla UC, do inglês *use case*, que significa caso de uso, seguida de um número sequencial (UC01 a UC07). Essa forma de identificação é comum na documentação de casos de uso e facilita a referência a cada funcionalidade ao longo do documento. A Figura 7 apresenta o diagrama.

Figura 7 – Diagrama de casos de uso do IntegraPSE



Fonte: Elaborado pelos autores (2026).

O caso de uso Validar entradas (UC07) é incluído («include») no cadastro, na pesquisa e na atualização, pois a validação sempre acontece nessas operações. O caso de uso Registrar participantes efetivos (UC05) estende («extend») a atualização da situação, pois só ocorre quando a ação é marcada como realizada.

3.2 Descrição dos casos de uso

A Tabela 2 apresenta a descrição resumida de cada caso de uso, com o fluxo principal e o principal fluxo alternativo.

Tabela 2 – Descrição dos casos de uso

Código	Caso de uso	Fluxo principal	Fluxo alternativo
UC01	Cadastrar ação	O operador escolhe a opção 1 e informa os dados da ação. O sistema valida os dados e grava a ação como planejada.	Se algum dado for inválido, o cadastro é cancelado e uma mensagem de erro é exibida.
UC02	Listar ações	O operador escolhe a opção 2 e o sistema exibe todas as ações cadastradas.	Se não houver ações, o sistema informa que nenhuma ação foi cadastrada.
UC03	Pesquisar ação	O operador escolhe o critério (código, escola ou tema), informa o valor e o sistema exibe as ações encontradas.	Se nenhuma ação corresponder ao critério, o sistema informa que nada foi encontrado.
UC04	Atualizar situação	O operador informa o código e a nova situação, e o sistema grava a alteração.	Se o código não existir ou a situação for inválida, nenhuma alteração é feita.
UC05	Registrar participantes efetivos	Quando a nova situação é REALIZADA, o operador informa o número de participantes efetivos.	Se o valor for negativo ou não numérico, a atualização é cancelada.
UC06	Gerar resumo geral	O operador escolhe a opção 5 e o sistema exibe as quantidades por situação e o percentual de participação.	Se não houver ações realizadas, o percentual é exibido como 0,0%.
UC07	Validar entradas	O sistema verifica tipo, preenchimento, formato e faixa de cada dado informado.	Se a entrada for inválida, a operação é cancelada sem alterar os registros.

Fonte: Elaborado pelos autores (2026).

3.3 Requisitos atualizados

Durante a implementação e os testes, alguns requisitos definidos na primeira entrega foram ajustados para tornar o sistema mais fácil de usar e mais seguro contra erros de digitação. Os demais requisitos permaneceram como descritos nas Tabelas 2 e 3 da primeira entrega. A Tabela 3 apresenta as alterações realizadas.

Tabela 3 – Alterações realizadas nos requisitos

Requisito	Alteração	Justificativa
Cadastrar ação (RF01)	A situação inicial passou a ser definida automaticamente como planejada.	Evitar que uma ação seja cadastrada como realizada sem participantes efetivos.
Pesquisar ação (RF03)	A pesquisa por escola e tema passou a aceitar parte do nome, sem diferenciar maiúsculas e minúsculas.	Nos testes, a pesquisa exata exigia digitar o nome completo da escola ou do tema.
Gerar resumo geral (RF06)	O resumo passou a mostrar também o total de ações cadastradas.	Facilitar a conferência dos valores apresentados.
Validar entradas (RF08)	Foram incluídas a validação da data e de textos digitados em campos numéricos.	Situações identificadas durante os testes.
Compatibilidade (RNF03)	As mensagens do programa foram escritas sem acentuação.	Garantir a exibição correta no console do Windows.

Fonte: Elaborado pelos autores (2026).

3.4 Testes realizados

Os testes foram realizados no console do Windows, a partir do programa compilado no Dev-C++, comparando o resultado obtido com o resultado esperado. Foram testadas as situações normais de uso e as entradas inválidas previstas nos requisitos. A Tabela 4 apresenta os casos de teste executados.

Tabela 4 – Casos de teste executados

ID	Caso de teste	Resultado esperado	Situação
CT01	Cadastro com dados válidos	Ação cadastrada como planejada	Aprovado
CT02	Cadastro com código repetido	Mensagem de código já existente	Aprovado
CT03	Cadastro com código negativo ou com letras	Mensagem de código inválido	Aprovado
CT04	Cadastro com campo obrigatório vazio	Cadastro cancelado	Aprovado
CT05	Cadastro com data inválida (31/02/2026)	Mensagem de data inválida	Aprovado
CT06	Cadastro com participantes negativos	Mensagem de quantidade inválida	Aprovado
CT07	Listagem sem ações cadastradas	Mensagem “Nenhuma acao cadastrada”	Aprovado
CT08	Listagem com quatro ações	Exibição das quatro ações	Aprovado
CT09	Pesquisa por parte do nome da escola	Exibição das ações da escola	Aprovado
CT10	Pesquisa por código inexistente	Mensagem de nenhuma ação encontrada	Aprovado
CT11	Atualizar ação para realizada	Situação alterada e participantes registrados	Aprovado
CT12	Atualizar ação inexistente	Mensagem de ação não encontrada	Aprovado

ID	Caso de teste	Resultado esperado	Situação
CT13	Resumo com ações em situações diferentes	Percentual de 90,0% (135 de 150)	Aprovado
CT14	Opção inexistente ou texto no menu	Mensagem de erro e novo menu	Aprovado
CT15	Encerramento do programa	Mensagem de encerramento	Aprovado

Fonte: Elaborado pelos autores (2026).

Todos os casos de teste foram aprovados na versão final. Durante os testes, foi identificada uma falha na leitura da data, que podia ultrapassar o tamanho do campo quando o usuário digitava caracteres a mais. A falha foi corrigida com a leitura da data em uma variável auxiliar, que só é copiada para a estrutura depois de validada.

3.5 Acompanhamento das sprints

A Tabela 5 apresenta o resultado das sprints realizadas após a primeira entrega, de acordo com o planejamento da seção 3.10 do primeiro documento.

Tabela 5 – Execução das sprints 07 a 14

Sprint	Objetivo	Resultado obtido	Situação
07	Estrutura inicial	Estrutura Acao, vetor, constantes e menu principal.	Concluída
08	Cadastro e listagem	Funções de cadastro e listagem com validações.	Concluída
09	Pesquisa e atualização	Pesquisa por código, escola e tema; atualização da situação.	Concluída
10	Resumo geral	Função de resumo e integração das funções ao menu.	Concluída
11	Casos de uso	Diagrama e descrição dos casos de uso.	Concluída
12	Testes e correções	Execução dos testes e correção da leitura da data.	Concluída
13	Documentação	Requisitos atualizados, manual, capturas de tela e README.	Concluída
14	Apresentação	Gravação do vídeo e revisão final dos entregáveis.	Concluída

Fonte: Elaborado pelos autores (2026).

4 PRODUÇÃO TEXTUAL — METEP

4.1 Manual do usuário

4.1.1 Objetivo do sistema

O IntegraPSE é uma aplicação de terminal que auxilia equipes de Saúde e Educação no registro e no acompanhamento das ações coletivas do Programa Saúde na Escola. Por meio do sistema, é possível cadastrar ações, consultá-las, atualizar sua situação, registrar a participação e gerar um resumo geral. O sistema deve ser utilizado apenas com dados coletivos ou fictícios.

4.1.2 Como executar o programa

Para executar o programa no Dev-C++, devem ser seguidos os passos abaixo:

- baixar o repositório do projeto no GitHub e abrir o arquivo src/integrapse.c no Dev-C++;
- selecionar a opção Execute > Compile & Run (tecla F11);
- utilizar o menu exibido no console, digitando o número da opção desejada e pressionando Enter.

Também é possível compilar pelo terminal com o comando gcc src/integrapse.c -o integrapse e executar o arquivo gerado.

4.1.3 Opções do menu

Tabela 6 – Opções do menu principal

Opção	Função	Como utilizar
1	Cadastrar ação	Informar código, escola, tema, data prevista (DD/MM/AAAA), público-alvo, responsável e participantes previstos.
2	Listar ações	Exibe todas as ações cadastradas.
3	Pesquisar ação	Escolher o critério (1 – código, 2 – escola ou 3 – tema) e digitar o valor procurado.
4	Atualizar situação	Informar o código e a nova situação (1 – planejada, 2 – realizada ou 3 – cancelada). Para ação realizada, informar os participantes efetivos.
5	Gerar resumo geral	Exibe as quantidades de ações por situação e o percentual de participação.
0	Encerrar	Finaliza o programa.

Fonte: Elaborado pelos autores (2026).

4.1.4 Observações

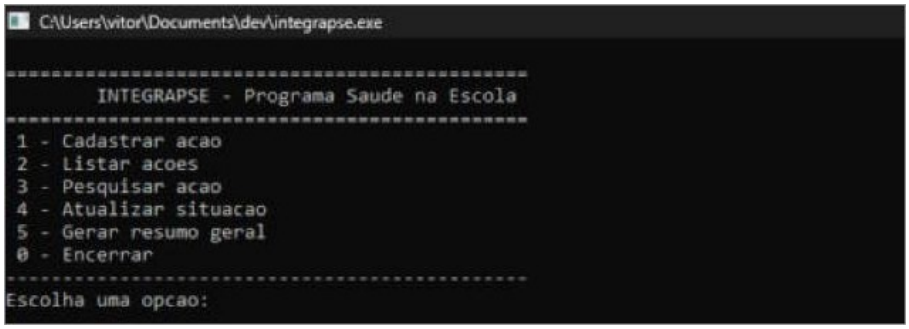
- Os dados ficam armazenados apenas durante a execução e são descartados ao encerrar o programa.

- O sistema permite cadastrar até 100 ações por execução.
- As mensagens iniciadas por [OK] indicam sucesso, e as iniciadas por [ERRO] indicam que nenhuma alteração foi feita.
- Não devem ser cadastrados nomes, diagnósticos ou outros dados pessoais de estudantes.

4.2 Capturas de tela

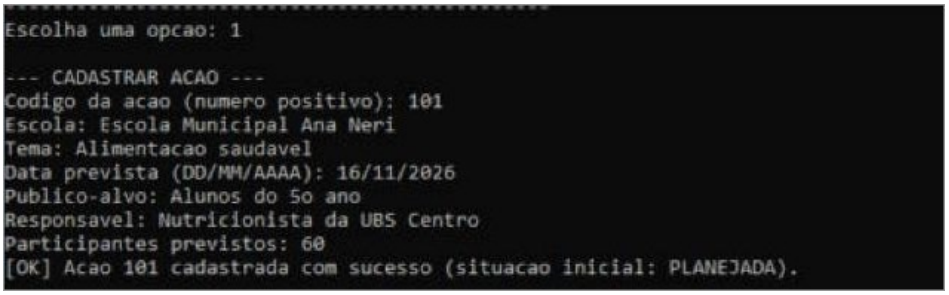
As figuras a seguir foram capturadas durante a execução do programa no console do Windows, a partir do Dev-C++. Para a demonstração, foram utilizadas quatro ações fictícias, cadastradas para as escolas Escola Municipal Ana Neri, Colégio Estadual João XXIII e Escola Estadual Santos Dumont.

Figura 8 – Menu principal do IntegraPSE



Fonte: Elaborado pelos autores (2026).

Figura 9 – Cadastro de uma ação com sucesso



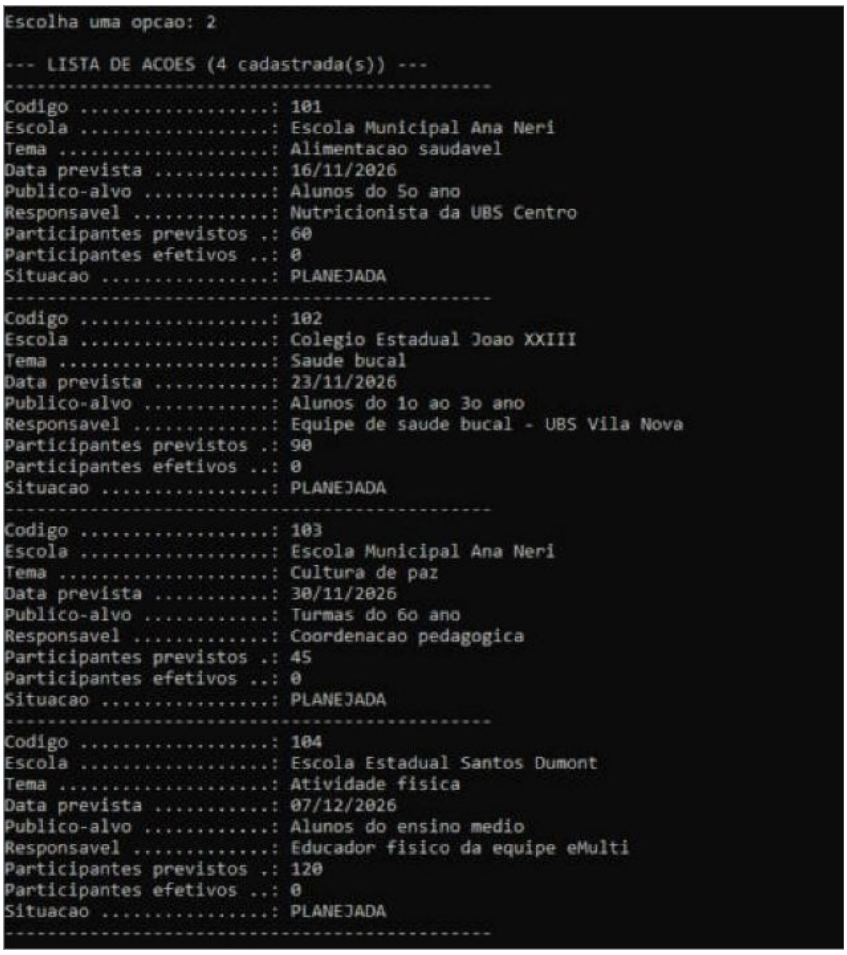
Fonte: Elaborado pelos autores (2026).

Figura 10 – Validações no cadastro: código repetido, data inválida, campo vazio e quantidade negativa



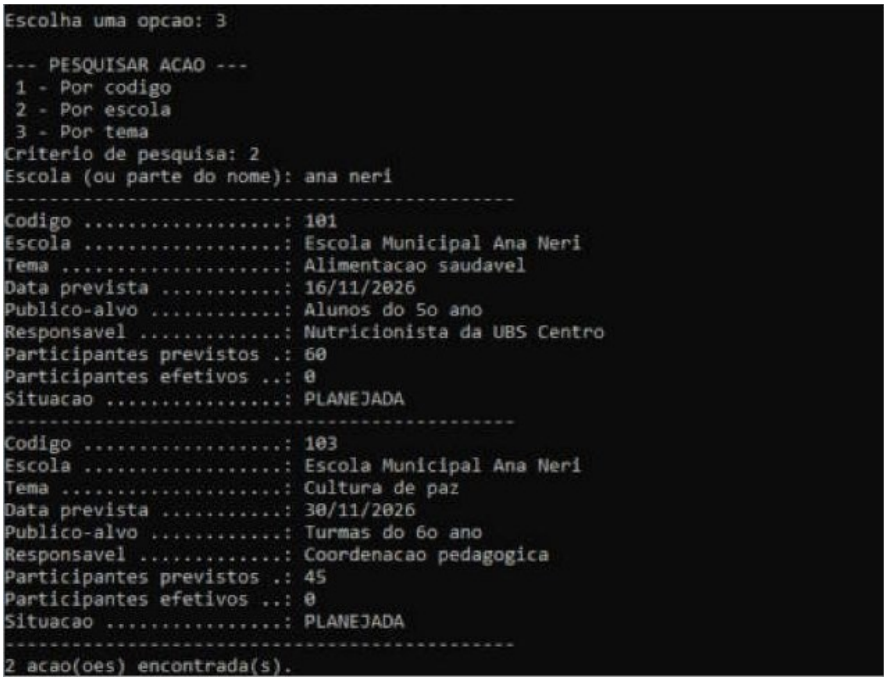
Fonte: Elaborado pelos autores (2026).

Figura 11 – Listagem das ações cadastradas



Fonte: Elaborado pelos autores (2026).

Figura 12 – Pesquisa por parte do nome da escola



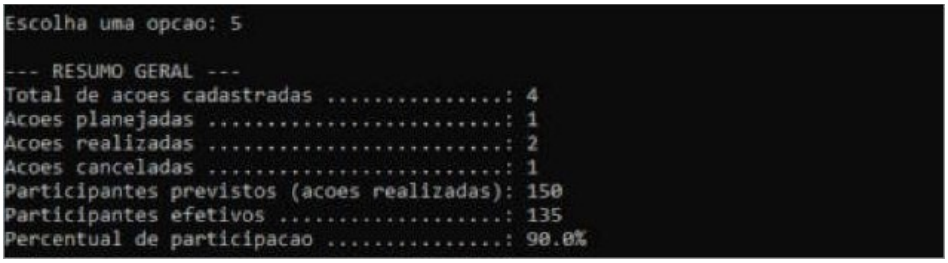
Fonte: Elaborado pelos autores (2026).

Figura 13 – Atualização da situação: ação realizada, ação cancelada e código inexistente



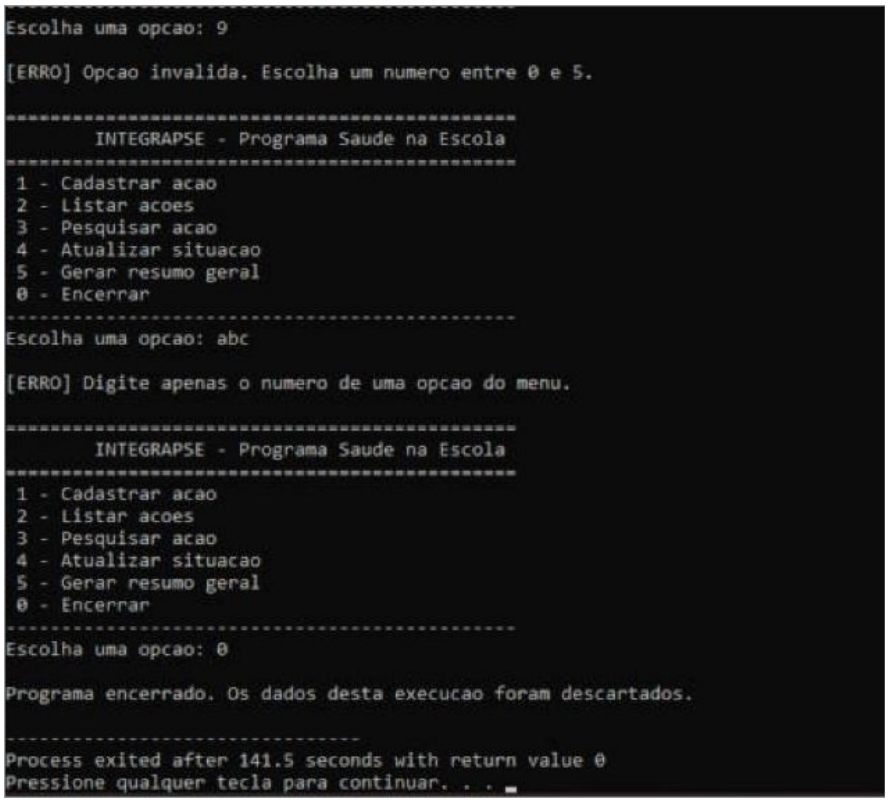
Fonte: Elaborado pelos autores (2026).

Figura 14 – Resumo geral das ações



Fonte: Elaborado pelos autores (2026).

Figura 15 – Opção inválida e encerramento do programa



Fonte: Elaborado pelos autores (2026).

5 CONSIDERAÇÕES FINAIS

O desenvolvimento do IntegraPSE permitiu responder à questão norteadora definida na primeira entrega. A aplicação em linguagem C possibilita planejar, registrar e acompanhar ações coletivas do Programa Saúde na Escola de forma organizada, utilizando apenas informações coletivas e preservando a privacidade dos estudantes. O sistema implementa todas as funcionalidades previstas e foi aprovado nos testes realizados.

A solução possui algumas limitações. Os dados ficam armazenados apenas durante a execução, não é possível excluir ações ou editar os demais campos de uma ação cadastrada e a interface em terminal é simples. Essas limitações estão de acordo com o escopo definido para o semestre.

Entre os principais aprendizados, destacamos a importância do planejamento realizado na primeira entrega. Os requisitos, fluxogramas e pseudocódigos facilitaram a implementação e a divisão das tarefas entre os integrantes. A etapa de testes também mostrou a importância de validar as entradas do usuário, e o uso de sprints semanais e do GitHub ajudou a organizar o trabalho em grupo.

Como melhorias futuras, o sistema poderia salvar os registros em arquivo, permitir a edição e a exclusão de ações, gerar resumos por escola ou período e, futuramente, contar com uma interface gráfica.

REPOSITÓRIO GITHUB

O repositório do projeto IntegraPSE contém o README, a identificação dos integrantes, o código-fonte completo na pasta src e a documentação produzida na primeira e na segunda etapas da AEP, organizada nas pastas docs/primeira-entrega e docs/segunda-entrega.

Link do repositório: https://github.com/felipezfroes/integra-pse

Link do vídeo de apresentação: **inserir aqui o link do vídeo com acesso liberado**

REFERÊNCIAS

BRASIL. Decreto nº 6.286, de 5 de dezembro de 2007. Institui o Programa Saúde na Escola – PSE e dá outras providências. Brasília, DF: Presidência da República, 2007. Disponível em: https://planalto.gov.br/ccivil_03/_ato2007-2010/2007/decreto/d6286.htm. Acesso em: 4 set. 2026.

BRASIL. Ministério da Saúde. **O que é o Programa Saúde na Escola (PSE)?** Brasília, DF, 2025a. Disponível em: https://www.gov.br/saude/pt-br/composicao/saps/pse/faq/o-que-e-o-programa. Acesso em: 4 set. 2026.

BRASIL. Ministério da Saúde. **Quais são as ações do PSE?** Brasília, DF, 2025b. Disponível em: https://www.gov.br/saude/pt-br/composicao/saps/pse/faq/quais-sao-as-acoes-do. Acesso em: 4 set. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. **Ciência & Saúde Coletiva**, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, jun. 2017. DOI: https://doi.org/10.1590/1413-81232017226.24262016. Disponível em: https://www.scielo.br/j/csc/a/nGRj8mdvvwZHvy6G76MrjfJ/. Acesso em: 4 set. 2026.

APÊNDICE A – CÓDIGO-FONTE COMPLETO DO INTEGRAPSE

Arquivo src/integrapse.c, versão final entregue no repositório do projeto.

```
/*
 * IntegraPSE - Sistema de Planejamento e Acompanhamento de Acoes de Saude na Escola
 * AEP 2o semestre de 2026 - Engenharia de Software - UniCesumar
 *
 * Aplicacao de terminal para registrar e acompanhar acoes coletivas do
 * Programa Saude na Escola (PSE). Utiliza apenas dados ficticios e coletivos.
 *
 * Compilacao: gcc integrapse.c -o integrapse
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#ifndef MAX_ACOES
#define MAX_ACOES 100
#endif
#define TAM_TEXTO 61
#define TAM_DATA 11
#define TAM_ENTRADA 128

#define SITUACAO_PLANEJADA 1
#define SITUACAO_REALIZADA 2
#define SITUACAO_CANCELADA 3

typedef struct {
    int codigo;
    char escola[TAM_TEXTO];
    char tema[TAM_TEXTO];
    char dataPrevista[TAM_DATA];
    char publicoAlvo[TAM_TEXTO];
    char responsavel[TAM_TEXTO];
    int participantesPrevistos;
    int participantesEfetivos;
    int situacao;
} Acao;

/* ---------- Prototipos ---------- */
void exibirMenu(void);
void lerTexto(const char *rotulo, char *destino, int tamanho);
int lerInteiro(const char *rotulo, int *valor);
int textoVazio(const char *texto);
int dataValida(const char *data);
int contemTexto(const char *texto, const char *trecho);
int buscarPorCodigo(const Acao acoes[], int quantidade, int codigo);
const char *nomeSituacao(int situacao);
void exibirAcao(const Acao *acao);

void cadastrarAcao(Acao acoes[], int *quantidade);
void listarAcoes(const Acao acoes[], int quantidade);
void pesquisarAcao(const Acao acoes[], int quantidade);
void atualizarSituacao(Acao acoes[], int quantidade);
```

```
void gerarResumo(const Acao acoes[], int quantidade);

/* ---------- Funcao principal ---------- */
int main(void) {
    Acao acoes[MAX_ACOES];
    int quantidadeAcoes = 0;
    int opcao;

    do {
        exibirMenu();

        if (!lerInteiro("Escolha uma opcao: ", &opcao)) {
            printf("\n[ERRO] Digite apenas o numero de uma opcao do menu.\n");
            opcao = -1;
            continue;
        }

        switch (opcao) {
            case 1:
                cadastrarAcao(acoes, &quantidadeAcoes);
                break;
            case 2:
                listarAcoes(acoes, quantidadeAcoes);
                break;
            case 3:
                pesquisarAcao(acoes, quantidadeAcoes);
                break;
            case 4:
                atualizarSituacao(acoes, quantidadeAcoes);
                break;
            case 5:
                gerarResumo(acoes, quantidadeAcoes);
                break;
            case 0:
                printf("\nPrograma encerrado. ");
                printf("Os dados desta execucao foram descartados.\n");
                break;
            default:
                printf("\n[ERRO] Opcao invalida. Escolha um numero entre 0 e 5.\n");
        }
    } while (opcao != 0);

    return 0;
}

/* ---------- Funcoes auxiliares de entrada e saida ---------- */
void exibirMenu(void) {
    printf("\n============================================\n");
    printf("        INTEGRAPSE - Programa Saude na Escola\n");
    printf("============================================\n");
    printf(" 1 - Cadastrar acao\n");
    printf(" 2 - Listar acoes\n");
    printf(" 3 - Pesquisar acao\n");
    printf(" 4 - Atualizar situacao\n");
    printf(" 5 - Gerar resumo geral\n");
    printf(" 0 - Encerrar\n");
    printf("--------------------------------------------\n");
}
```

```
/* Le uma linha inteira, remove a quebra de linha e descarta o excedente. */
void lerTexto(const char *rotulo, char *destino, int tamanho) {
    int c;

    printf("%s", rotulo);
    if (fgets(destino, tamanho, stdin) == NULL) {
        destino[0] = '\0';
        return;
    }

    if (strchr(destino, '\n') != NULL) {
        destino[strcspn(destino, "\n")] = '\0';
    } else {
        while ((c = getchar()) != '\n' && c != EOF) {
        }
    }

    if (strlen(destino) > 0 && destino[strlen(destino) - 1] == '\r') {
        destino[strlen(destino) - 1] = '\0';
    }
}

/* Retorna 1 se a entrada for um numero inteiro valido e 0 caso contrario. */
int lerInteiro(const char *rotulo, int *valor) {
    char entrada[TAM_ENTRADA];
    char *fim;
    long numero;

    lerTexto(rotulo, entrada, TAM_ENTRADA);
    if (textoVazio(entrada)) {
        return 0;
    }

    numero = strtol(entrada, &fim, 10);
    while (isspace((unsigned char)*fim)) {
        fim++;
    }
    if (*fim != '\0' || numero > 999999 || numero < -999999) {
        return 0;
    }

    *valor = (int)numero;
    return 1;
}

int textoVazio(const char *texto) {
    while (*texto != '\0') {
        if (!isspace((unsigned char)*texto)) {
            return 0;
        }
        texto++;
    }
    return 1;
}

/* Aceita o formato DD/MM/AAAA e confere dia, mes e ano bissexto. */
int dataValida(const char *data) {
```

```
    int dia, mes, ano, i;
    int diasNoMes[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    if (strlen(data) != 10 || data[2] != '/' || data[5] != '/') {
        return 0;
    }
    for (i = 0; i < 10; i++) {
        if (i != 2 && i != 5 && !isdigit((unsigned char)data[i])) {
            return 0;
        }
    }

    dia = (data[0] - '0') * 10 + (data[1] - '0');
    mes = (data[3] - '0') * 10 + (data[4] - '0');
    ano = atoi(&data[6]);

    if (ano < 2000 || ano > 2100 || mes < 1 || mes > 12) {
        return 0;
    }
    if ((ano % 4 == 0 && ano % 100 != 0) || ano % 400 == 0) {
        diasNoMes[1] = 29;
    }
    return dia >= 1 && dia <= diasNoMes[mes - 1];
}

/* Pesquisa por trecho, sem diferenciar letras maiusculas e minusculas. */
int contemTexto(const char *texto, const char *trecho) {
    int i, j;
    int tamTexto = (int)strlen(texto);
    int tamTrecho = (int)strlen(trecho);

    for (i = 0; i <= tamTexto - tamTrecho; i++) {
        for (j = 0; j < tamTrecho; j++) {
            if (tolower((unsigned char)texto[i + j]) !=
                tolower((unsigned char)trecho[j])) {
                break;
            }
        }
        if (j == tamTrecho) {
            return 1;
        }
    }
    return 0;
}

/* Retorna a posicao da acao no vetor ou -1 se o codigo nao existir. */
int buscarPorCodigo(const Acao acoes[], int quantidade, int codigo) {
    int i;
    for (i = 0; i < quantidade; i++) {
        if (acoes[i].codigo == codigo) {
            return i;
        }
    }
    return -1;
}

const char *nomeSituacao(int situacao) {
    switch (situacao) {
```

```
        case SITUACAO_PLANEJADA: return "PLANEJADA";
        case SITUACAO_REALIZADA: return "REALIZADA";
        case SITUACAO_CANCELADA: return "CANCELADA";
        default: return "DESCONHECIDA";
    }
}

void exibirAcao(const Acao *acao) {
    printf("---------------------------------------------\n");
    printf("Codigo ..................: %d\n", acao->codigo);
    printf("Escola ..................: %s\n", acao->escola);
    printf("Tema ....................: %s\n", acao->tema);
    printf("Data prevista ...........: %s\n", acao->dataPrevista);
    printf("Publico-alvo ............: %s\n", acao->publicoAlvo);
    printf("Responsavel .............: %s\n", acao->responsavel);
    printf("Participantes previstos .: %d\n", acao->participantesPrevistos);
    printf("Participantes efetivos ..: %d\n", acao->participantesEfetivos);
    printf("Situacao ................: %s\n", nomeSituacao(acao->situacao));
}

/* ---------- Funcionalidades do sistema ---------- */
void cadastrarAcao(Acao acoes[], int *quantidade) {
    Acao nova;
    char data[TAM_ENTRADA];

    printf("\n--- CADASTRAR ACAO ---\n");

    if (*quantidade >= MAX_ACOES) {
        printf("[ERRO] Limite de %d acoes atingido. ", MAX_ACOES);
        printf("Nao e possivel cadastrar.\n");
        return;
    }

    if (!lerInteiro("Codigo da acao (numero positivo): ", &nova.codigo) ||
        nova.codigo <= 0) {
        printf("[ERRO] Codigo invalido. Informe um numero inteiro maior que zero.\n");
        return;
    }
    if (buscarPorCodigo(acoes, *quantidade, nova.codigo) != -1) {
        printf("[ERRO] Ja existe uma acao com o codigo %d.\n", nova.codigo);
        return;
    }

    lerTexto("Escola: ", nova.escola, TAM_TEXTO);
    lerTexto("Tema: ", nova.tema, TAM_TEXTO);
    lerTexto("Data prevista (DD/MM/AAAA): ", data, TAM_ENTRADA);
    lerTexto("Publico-alvo: ", nova.publicoAlvo, TAM_TEXTO);
    lerTexto("Responsavel: ", nova.responsavel, TAM_TEXTO);

    if (textoVazio(nova.escola) || textoVazio(nova.tema) ||
        textoVazio(nova.publicoAlvo) || textoVazio(nova.responsavel)) {
        printf("[ERRO] Existem campos obrigatorios vazios. Cadastro cancelado.\n");
        return;
    }
    if (!dataValida(data)) {
        printf("[ERRO] Data invalida. Use DD/MM/AAAA. Cadastro cancelado.\n");
        return;
    }
```

```
    strcpy(nova.dataPrevista, data);
    if (!lerInteiro("Participantes previstos: ", &nova.participantesPrevistos) ||
        nova.participantesPrevistos < 0) {
        printf("[ERRO] Quantidade de participantes invalida. Cadastro cancelado.\n");
        return;
    }

    nova.participantesEfetivos = 0;
    nova.situacao = SITUACAO_PLANEJADA;

    acoes[*quantidade] = nova;
    (*quantidade)++;

    printf("[OK] Acao %d cadastrada com sucesso ", nova.codigo);
    printf("(situacao inicial: PLANEJADA).\n");
}

void listarAcoes(const Acao acoes[], int quantidade) {
    int i;

    printf("\n--- LISTA DE ACOES (%d cadastrada(s)) ---\n", quantidade);

    if (quantidade == 0) {
        printf("Nenhuma acao cadastrada.\n");
        return;
    }

    for (i = 0; i < quantidade; i++) {
        exibirAcao(&acoes[i]);
    }
    printf("-----------------------------------------------\n");
}

void pesquisarAcao(const Acao acoes[], int quantidade) {
    int opcao, codigo, i, encontrados = 0;
    char termo[TAM_TEXTO];

    printf("\n--- PESQUISAR ACAO ---\n");

    if (quantidade == 0) {
        printf("Nenhuma acao cadastrada para pesquisar.\n");
        return;
    }

    printf(" 1 - Por codigo\n 2 - Por escola\n 3 - Por tema\n");
    if (!lerInteiro("Criterio de pesquisa: ", &opcao) || opcao < 1 || opcao > 3) {
        printf("[ERRO] Criterio de pesquisa invalido.\n");
        return;
    }

    if (opcao == 1) {
        if (!lerInteiro("Codigo procurado: ", &codigo)) {
            printf("[ERRO] Codigo invalido.\n");
            return;
        }
        i = buscarPorCodigo(acoes, quantidade, codigo);
        if (i != -1) {
            exibirAcao(&acoes[i]);
```

```
            encontrados = 1;
        }
    } else {
        if (opcao == 2) {
            lerTexto("Escola (ou parte do nome): ", termo, TAM_TEXTO);
        } else {
            lerTexto("Tema (ou parte do nome): ", termo, TAM_TEXTO);
        }
        if (textoVazio(termo)) {
            printf("[ERRO] O termo de pesquisa nao pode ficar vazio.\n");
            return;
        }
        for (i = 0; i < quantidade; i++) {
            const char *campo = (opcao == 2) ? acoes[i].escola : acoes[i].tema;
            if (contemTexto(campo, termo)) {
                exibirAcao(&acoes[i]);
                encontrados++;
            }
        }
    }

    if (encontrados == 0) {
        printf("Nenhuma acao encontrada para o criterio informado.\n");
    } else {
        printf("------------------------------------------------\n");
        printf("%d acao(oes) encontrada(s).\n", encontrados);
    }
}

void atualizarSituacao(Acao acoes[], int quantidade) {
    int codigo, posicao, novaSituacao, efetivos;

    printf("\n--- ATUALIZAR SITUACAO ---\n");

    if (quantidade == 0) {
        printf("Nenhuma acao cadastrada.\n");
        return;
    }

    if (!lerInteiro("Codigo da acao: ", &codigo)) {
        printf("[ERRO] Codigo invalido.\n");
        return;
    }
    posicao = buscarPorCodigo(acoes, quantidade, codigo);
    if (posicao == -1) {
        printf("[ERRO] Acao com codigo %d nao encontrada.\n", codigo);
        return;
    }

    printf("Situacao atual: %s\n", nomeSituacao(acoes[posicao].situacao));
    printf(" 1 - Planejada\n 2 - Realizada\n 3 - Cancelada\n");
    if (!lerInteiro("Nova situacao: ", &novaSituacao) ||
        novaSituacao < 1 || novaSituacao > 3) {
        printf("[ERRO] Situacao invalida. Nenhuma alteracao foi feita.\n");
        return;
    }

    if (novaSituacao == SITUACAO_REALIZADA) {
```

```
        if (!lerInteiro("Participantes efetivos: ", &efetivos) || efetivos < 0) {
            printf("[ERRO] Quantidade invalida. Nenhuma alteracao foi feita.\n");
            return;
        }
        acoes[posicao].participantesEfetivos = efetivos;
    } else {
        acoes[posicao].participantesEfetivos = 0;
    }

    acoes[posicao].situacao = novaSituacao;
    printf("[OK] Acao %d atualizada para %s.\n", codigo, nomeSituacao(novaSituacao));
}

void gerarResumo(const Acao acoes[], int quantidade) {
    int i;
    int planejadas = 0, realizadas = 0, canceladas = 0;
    int previstosRealizadas = 0, totalEfetivos = 0;
    float percentual = 0.0f;

    printf("\n--- RESUMO GERAL ---\n");

    if (quantidade == 0) {
        printf("Nenhuma acao cadastrada.\n");
        return;
    }

    for (i = 0; i < quantidade; i++) {
        if (acoes[i].situacao == SITUACAO_PLANEJADA) {
            planejadas++;
        } else if (acoes[i].situacao == SITUACAO_REALIZADA) {
            realizadas++;
            previstosRealizadas += acoes[i].participantesPrevistos;
            totalEfetivos += acoes[i].participantesEfetivos;
        } else if (acoes[i].situacao == SITUACAO_CANCELADA) {
            canceladas++;
        }
    }

    if (previstosRealizadas > 0) {
        percentual = (float)totalEfetivos / previstosRealizadas * 100.0f;
    }

    printf("Total de acoes cadastradas ...............: %d\n", quantidade);
    printf("Acoes planejadas .........................: %d\n", planejadas);
    printf("Acoes realizadas .........................: %d\n", realizadas);
    printf("Acoes canceladas .........................: %d\n", canceladas);
    printf("Participantes previstos (acoes realizadas): %d\n", previstosRealizadas);
    printf("Participantes efetivos ...................: %d\n", totalEfetivos);
    printf("Percentual de participacao ...............: %.1f%%\n", percentual);
}
```